9. 성능 최적화와 메트릭

9.1 성능 최적화 개요

Narwhal 시스템의 성능 최적화는 처리량(Throughput), 지연시간(Latency), 메모리 사용량, CPU 효율성 등 다양한 측면에서 이루어집니다.

9.1.1 최적화 목표

- **높은 처리량**: 초당 수만 건의 트랜잭션 처리
- **낮은 지연시간**: 수백 밀리초 내 컨센서스 완료
- **메모리 효율성**: 제한된 메모리로 대용량 DAG 관리
- **네트워크 효율성**: 최소한의 네트워크 오버헤드

9.1.2 성능 메트릭 체계

```
pub struct PerformanceMetrics {
// 처리량 메트릭
pub throughput_metrics: ThroughputMetrics,
// 지연시간 메트릭
pub latency_metrics: LatencyMetrics,
// 리소스 사용량 메트릭
pub resource_metrics: ResourceMetrics,
// 네트워크 메트릭
```

```
pub network_metrics: NetworkMetrics,

}
```

9.2 컨센서스 성능 최적화

9.2.1 DAG 크기 제한

```
impl ConsensusState {

pub fn update(&mut self, certificate: &Certificate, gc_depth: Round) {

// 최신 커밋 라운드 업데이트

self.last_committed

.entry(certificate.origin())

.and_modify(|r| *r = max(*r, certificate.round()))

.or_insert_with(|| certificate.round());


let last_committed_round = *std::iter::Iterator::max(self.last_committed.values

self.last_committed_round = last_committed_round;


// 메트릭 업데이트

self.metrics

.last_committed_round

.with_label_values(&[])

.set(last_committed_round as i64);
```

```
// GC 깊이를 초과한 인증서 제거
self.dag.retain(|r, _| r + gc_depth >= last_committed_round);
// 개별 노드의 오래된 인증서 제거
for (name, round) in &self.last_committed {
self.dag.retain(|r, authorities| {
if  r < round {
authorities.remove(name);
}
!authorities.is_empty()
});
}
}
}
```

9.2.2 배치 처리 최적화

```
pub  struct BatchProcessor {
max_batch_size: usize,
max_batch_delay: Duration,
compression_enabled: bool,
deduplication_enabled: bool,
}
```

```
impl BatchProcessor {
pub fn optimize_batch_creation(&self, transactions: Vec<Transaction>) -> Optim
let mut optimized_txs = transactions;
// 중복 제거
if self.deduplication_enabled {
optimized_txs = self.remove_duplicate_transactions(optimized_txs);
}
// 트랜잭션 정렬 (가스비 기준)
optimized_txs.sort_by(|a, b| b.gas_price().cmp(&a.gas_price()));
// 배치 크기 최적화
if optimized_txs.len() * AVERAGE_TX_SIZE > self.max_batch_size {
optimized_txs.truncate(self.max_batch_size / AVERAGE_TX_SIZE);
}
OptimizedBatch::new(optimized_txs, self.compression_enabled)
}
fn remove_duplicate_transactions(&self, mut transactions: Vec<Transaction>) ->
let mut seen = HashSet::new();
transactions.retain(|tx| seen.insert(tx.digest()));
transactions
}
}
```

9.3 메모리 최적화

9.3.1 스마트 캐싱

```
pub struct SmartCache<K, V> {
hot_cache: LruCache<K, V>, // 자주 접근하는 데이터
warm_cache: LruCache<K, V>, // 가끔 접근하는 데이터
access_counter: HashMap<K, u64>,
promotion_threshold: u64,
}

impl<K: Clone + Hash + Eq, V: Clone> SmartCache<K, V> {
pub fn get(&mut self, key: &K) -> Option<V> {
// 접근 횟수 증가
*self.access_counter.entry(key.clone()).or_insert(0) += 1;
// Hot 캐시에서 먼저 확인
if let Some(value) = self.hot_cache.get(key) {
return Some(value.clone());
}
// Warm 캐시에서 확인
if let Some(value) = self.warm_cache.get(key) {
// 접근 빈도가 높으면 Hot 캐시로 승격
if self.access_counter.get(key).unwrap_or(&0) > &self.promotion_threshold {
```

```
self.hot_cache.put(key.clone(), value.clone());
self.warm_cache.pop(key);
}
return Some(value);
}
None
}
pub fn put(&mut self, key: K, value: V) {
// 접근 빈도에 따라 적절한 캐시에 저장
let access_count = self.access_counter.get(&key).unwrap_or(&0);
if *access_count > self.promotion_threshold {
self.hot_cache.put(key, value);
} else {
self.warm_cache.put(key, value);
}
}
}
```

9.3.2 메모리 풀링

```
pub struct MemoryPool<T> {
pool: Vec<T>,
factory: Box<dyn Fn() -> T>,
```

```
max_size: usize,
}

impl<T> MemoryPool<T> {
pub fn new<F>(factory: F, max_size: usize) -> Self
where
F: Fn() -> T + 'static,
{
MemoryPool {
pool: Vec::with_capacity(max_size),
factory: Box::new(factory),
max_size,
}
}
pub fn acquire(&mut self) -> PooledObject<T> {
let object = self.pool.pop().unwrap_or_else(|| (self.factory)());
PooledObject::new(object, self)
}
pub fn release(&mut self, object: T) {
if self.pool.len() < self.max_size {
self.pool.push(object);
}
// 최대 크기 초과 시 객체 버림 (GC가 처리)
```

```
}
}

pub struct PooledObject<'a, T> {
object: Option<T>,
pool: &'a mut MemoryPool<T>,
}

impl<'a, T> Drop for PooledObject<'a, T> {
fn drop(&mut self) {
if let Some(object) = self.object.take() {
self.pool.release(object);
}
}
}
```

9.4 네트워크 최적화

9.4.1 메시지 압축

```
pub struct MessageCompressor {
compression_algorithm: CompressionAlgorithm,
compression_threshold: usize,
```

```
compression_level: u32,
}


impl MessageCompressor {
pub  fn  compress_message(&self, message: &[u8]) -> Result<CompressedMessage, Co
if  message.len() < self.compression_threshold {
return Ok(CompressedMessage::Uncompressed(message.to_vec()));
}
let  compressed = match  self.compression_algorithm {
CompressionAlgorithm::Gzip => {
self.gzip_compress(message, self.compression_level)?
}
CompressionAlgorithm::Lz4 => {
self.lz4_compress(message)?
}
CompressionAlgorithm::Zstd => {
self.zstd_compress(message, self.compression_level)?
}
};
// 압축 효과가 있는 경우만 압축된 버전 사용
if  compressed.len() < message.len() * 9 / 10 { // 10% 이상 절약
Ok(CompressedMessage::Compressed {
algorithm: self.compression_algorithm,
```

```
data: compressed,
original_size: message.len(),
})
} else {
Ok(CompressedMessage::Uncompressed(message.to_vec()))
}
}
}
```

9.4.2 연결 풀 최적화

```
pub  struct OptimizedConnectionPool {
connections: HashMap<SocketAddr, ConnectionSlot>,
connection_config: ConnectionConfig,
health_checker: HealthChecker,
load_balancer: LoadBalancer,
}

struct ConnectionSlot {
connections: Vec<Arc<Connection>>,
round_robin_index: AtomicUsize,
last_health_check: Instant,
}
```

```
impl OptimizedConnectionPool {
pub  async  fn  get_connection(&mut  self, addr: SocketAddr) -> Result<Arc<Conne
let  slot = self.connections.entry(addr).or_insert_with(|| {
ConnectionSlot {
connections: Vec::new(),
round_robin_index: AtomicUsize::new(0),
last_health_check: Instant::now(),
}
});
// 건강한 연결 찾기
for  _  in  0..slot.connections.len() {
let  index = slot.round_robin_index.fetch_add(1, Ordering::Relaxed) % slot.conne
let  connection = &slot.connections[index];
if  connection.is_healthy().await {
return Ok(connection.clone());
}
}
// 새 연결 생성
let  new_connection = self.create_optimized_connection(addr).await?;
let  arc_connection = Arc::new(new_connection);
slot.connections.push(arc_connection.clone());
Ok(arc_connection)
```

```
}

async fn create_optimized_connection(&self, addr: SocketAddr) -> Result<Connec

let mut stream = TcpStream::connect(addr).await?;

// TCP 최적화 설정

stream.set_nodelay(true)?; // Nagle 알고리즘 비활성화

stream.set_keepalive(Some(Duration::from_secs(30)))?; // Keep-alive 설정

// 버퍼 크기 최적화

let socket = socket2::Socket::from(stream);

socket.set_send_buffer_size(self.connection_config.send_buffer_size)?;

socket.set_recv_buffer_size(self.connection_config.recv_buffer_size)?;

Ok(Connection::new(socket.into(), self.connection_config.clone()))

}

}
```

9.5 CPU 최적화

9.5.1 병렬 서명 검증

```
pub struct ParallelSignatureVerifier {

thread_pool: ThreadPool,

batch_size: usize,

}
```

```
impl ParallelSignatureVerifier {
pub  async  fn  verify_signatures_parallel(
&self,
signatures: Vec<(PublicKey, Signature, Vec<u8>)>
) -> Result<Vec<bool>, VerificationError> {
let  chunks: Vec<_> = signatures.chunks(self.batch_size).collect();
let  mut  handles = Vec::new();
for  chunk  in  chunks {
let  chunk = chunk.to_vec();
let  handle = self.thread_pool.spawn(async  move {
chunk.into_iter()
.map(|(public_key, signature, message)| {
public_key.verify(&message, &signature).is_ok()
})
.collect::<Vec<bool>>()
});
handles.push(handle);
}
// 모든 결과 수집
let  mut  results = Vec::new();
for  handle  in  handles {
let  chunk_results = handle.await?;
results.extend(chunk_results);
```

```
}
Ok(results)
}
}
```

9.5.2 SIMD 최적화

```
#[cfg(target_arch = "x86_64")]
use std::arch::x86_64::*;

pub struct SIMDHasher {
_phantom: std::marker::PhantomData<()>,
}

impl SIMDHasher {
#[cfg(target_arch = "x86_64")]
pub unsafe fn hash_batch_avx2(data: &[&[u8]]) -> Vec<[u8; 32]> {
let mut results = Vec::with_capacity(data.len());
// AVX2를 사용한 병렬 해싱
for chunk in data.chunks(8) { // AVX2는 8개 병렬 처리
let mut hashes = [[0u8; 32]; 8];
for (i, &item) in chunk.iter().enumerate() {
// SIMD 명령어를 사용한 해싱
```

```
let hash = self.simd_hash(item);
hashes[i] = hash;
}
results.extend_from_slice(&hashes[..chunk.len()]);
}
results
}
#[cfg(target_arch = "x86_64")]
unsafe fn simd_hash(&self, data: &[u8]) -> [u8; 32] {
// SIMD 명령어를 사용한 최적화된 해싱 구현
// 실제로는 blake2b나 sha256의 SIMD 구현을 사용
blake2b_simd::blake2b(data).as_bytes().try_into().unwrap()
}
}
```

9.6 메트릭 수집 시스템

9.6.1 컨센서스 메트릭

```
use prometheus::{Counter, Histogram, Gauge, Registry};

pub struct ConsensusMetrics {
// 처리량 메트릭
```

```
pub certificates_processed: Counter,
pub transactions_committed: Counter,
pub batches_processed: Counter,
// 지연시간 메트릭
pub consensus_latency: Histogram,
pub certificate_creation_time: Histogram,
pub dag_traversal_time: Histogram,
// 상태 메트릭
pub dag_size_bytes: Gauge,
pub consensus_dag_rounds: Gauge,
pub last_committed_round: Gauge,
// 에러 메트릭
pub consensus_errors: Counter,
pub sync_failures: Counter,
pub signature_verification_failures: Counter,
}

impl ConsensusMetrics {
pub fn new(registry: &Registry) -> Self {
let certificates_processed = Counter::new(
"narwhal_certificates_processed_total",
"Total number of certificates processed"
).expect("Failed to create certificates_processed counter");
```

```
let consensus_latency = Histogram::with_opts(
prometheus::HistogramOpts::new(
"narwhal_consensus_latency_seconds",
"Time taken for consensus to complete"
).buckets(vec![0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0])
).expect("Failed to create consensus_latency histogram");
let dag_size_bytes = Gauge::new(
"narwhal_dag_size_bytes",
"Current size of the DAG in bytes"
).expect("Failed to create dag_size_bytes gauge");
// 메트릭 등록
registry.register(Box::new(certificates_processed.clone())).unwrap();
registry.register(Box::new(consensus_latency.clone())).unwrap();
registry.register(Box::new(dag_size_bytes.clone())).unwrap();
ConsensusMetrics {
certificates_processed,
consensus_latency,
dag_size_bytes,
// ... 다른 메트릭들
}
}
pub fn record_certificate_processed(&self) {
self.certificates_processed.inc();
```

```
}
pub fn record_consensus_latency(&self, duration: Duration) {
self.consensus_latency.observe(duration.as_secs_f64());
}
pub fn update_dag_size(&self, size_bytes: usize) {
self.dag_size_bytes.set(size_bytes as f64);
}
}
```

9.6.2 네트워크 메트릭

```
pub struct NetworkMetrics {
// 메시지 메트릭
pub messages_sent: Counter,
pub messages_received: Counter,
pub bytes_sent: Counter,
pub bytes_received: Counter,
// 연결 메트릭
pub active_connections: Gauge,
pub connection_failures: Counter,
pub connection_latency: Histogram,
// 대역폭 메트릭
pub bandwidth_utilization: Gauge,
```

```
pub  message_compression_ratio: Histogram,
}


impl NetworkMetrics {
pub  fn  record_message_sent(&self, size_bytes: usize) {
self.messages_sent.inc();
self.bytes_sent.inc_by(size_bytes  as u64);
}
pub  fn  record_compression_ratio(&self, original_size: usize, compressed_size:
let  ratio = compressed_size  as f64 / original_size  as f64;
self.message_compression_ratio.observe(ratio);
}
}
```

9.7 성능 모니터링 대시보드

9.7.1 실시간 성능 추적

```
pub  struct PerformanceDashboard {
metrics_collector: MetricsCollector,
alert_manager: AlertManager,
dashboard_config: DashboardConfig,
}
```

```
impl PerformanceDashboard {
pub  async  fn  start_monitoring(&mut  self) {
let  mut  interval = tokio::time::interval(Duration::from_secs(1));
loop {
interval.tick().await;
// 메트릭 수집
let  current_metrics = self.metrics_collector.collect_current_metrics().await;
// 성능 임계값 확인
self.check_performance_thresholds(&current_metrics).await;
// 대시보드 업데이트
self.update_dashboard(&current_metrics).await;
}
}
async  fn  check_performance_thresholds(&mut  self, metrics: &PerformanceSnapsho
// TPS 임계값 확인
if  metrics.transactions_per_second < self.dashboard_config.min_tps_threshold {
self.alert_manager.send_alert(Alert::LowThroughput {
current_tps: metrics.transactions_per_second,
threshold: self.dashboard_config.min_tps_threshold,
}).await;
}
// 지연시간 임계값 확인
```

```
if metrics.average_latency > self.dashboard_config.max_latency_threshold {
self.alert_manager.send_alert(Alert::HighLatency {
current_latency: metrics.average_latency,
threshold: self.dashboard_config.max_latency_threshold,
}).await;
}
// 메모리 사용량 확인
if metrics.memory_usage_percent > self.dashboard_config.max_memory_threshold {
self.alert_manager.send_alert(Alert::HighMemoryUsage {
current_usage: metrics.memory_usage_percent,
threshold: self.dashboard_config.max_memory_threshold,
}).await;
}
}
}
```

9.7.2 성능 분석 도구

```
pub struct PerformanceAnalyzer {
historical_data: VecDeque<PerformanceSnapshot>,
analysis_window: Duration,
}
```

```rust
impl PerformanceAnalyzer {
pub fn analyze_performance_trends(&self) -> PerformanceAnalysis {
let recent_data: Vec<_> = self.historical_data
.iter()
.filter(|snapshot| {
snapshot.timestamp > SystemTime::now() - self.analysis_window
})
.collect();
PerformanceAnalysis {
throughput_trend: self.calculate_throughput_trend(&recent_data),
latency_trend: self.calculate_latency_trend(&recent_data),
resource_utilization: self.calculate_resource_utilization(&recent_data),
bottleneck_analysis: self.identify_bottlenecks(&recent_data),
recommendations: self.generate_recommendations(&recent_data),
}
}
fn identify_bottlenecks(&self, data: &[&PerformanceSnapshot]) -> Vec<Bottleneck
let mut bottlenecks = Vec::new();
// CPU 병목 확인
let avg_cpu_usage = data.iter()
.map(|s| s.cpu_usage_percent)
.sum::<f64>() / data.len() as f64;
if avg_cpu_usage > 80.0 {
```

```
bottlenecks.push(Bottleneck::CPU {
usage_percent: avg_cpu_usage,
recommendation: "Consider optimizing CPU-intensive operations or scaling horizon
});
}
// 네트워크 병목 확인
let avg_network_utilization = data.iter()
.map(|s| s.network_utilization_percent)
.sum::<f64>() / data.len() as f64;
if avg_network_utilization > 70.0 {
bottlenecks.push(Bottleneck::Network {
utilization_percent: avg_network_utilization,
recommendation: "Consider message compression or network optimization".to_string
});
}
bottlenecks
}
}
```

이러한 성능 최적화와 메트릭 시스템을 통해 Narwhal은 높은 성능을 유지하면서도 시스템 상태를 실시간으로 모니터링할 수 있습니다.